

Oscar — Werking van de CTA's & kelnerselectie

Oscar is de slimme laag boven de werkvloer. Hij kijkt mee met wat er gebeurt — seaten, bestellen, drukte — en stuurt op het juiste moment één concreet zetje (een **CTA**, call-to-action) naar de handy van de juiste kelner. Dit document legt uit hoe de CTA's werken en hoe Oscar bepaalt naar welke kelner een melding gaat.

1. Hoe een CTA werkt

De handy van de kelner is een "**dom**" **scherm**: hij toont alleen wat Oscar erop pusht. Alle logica zit in de backend, per locatie aan/uit te zetten. Een CTA is een kaartje met een titel en knoppen; de kelner handelt het af, Oscar registreert de respons.

- **Eén melding, één handy.** Elke CTA gaat naar precies één kelner (zie §2). Liever geen melding dan de verkeerde.
- **Max. 3 kaarten per handy.** Zijn er meer, dan bepaalt de prioriteit welke 3 zichtbaar zijn; de rest staat in de wachtrij. Volgorde: 1, 2, 9, 10, 3, 4, 5, 6, 7, 8 (CTA 9/10 vlak na 1/2 omdat de gast wacht).
- **Afhandelen is verplicht.** Pas na afhandeling of verval schuift de volgende CTA door.
- **Ticket afgerekend = alle CTA's voor die tafel direct weg.**
- **Een tafel sluiten is altijd een kelner-beslissing** — geen enkele CTA of melding doet dat automatisch.
- **Geen onderbreking van een drukke kelner.** Zolang de kelner net iets deed (order / betalen / wachten) wacht Oscar even; CTA's komen pas op een rustig moment. Per CTA in/uit te schakelen ("block by busy"); CTA 3 is uitgezonderd. Staat los van de venue-brede drukte.
- **Vibratie** staat per CTA in de backend aan/uit — de handy trilt bij een push als het aanstaat (niet getoond op de kaart).
- **Niet overspoelen + meten of het werkt.** Max. 3 per handy, niets naar een drukke kelner, en per CTA meet Oscar via de log of het gewenste effect volgt. Een CTA die structureel genegeerd wordt, gaat eruit of wordt vervangen — liever een paar effectieve CTA's dan veel ruis.
- **Gefaseerde introductie (CTA-niveau 1–3).** Eén knop per locatie bepaalt hoeveel CTA's aan staan én hoe streng de rem. Start op niveau 1 (alleen kern-CTA's, strenge rem) zodat kelners het accepteren; schaal later op. Nog-uitgeschakelde CTA's worden wél shadow-geloggd, zodat je ziet wanneer opschalen zinvol is.
- **Knoppen: één taal, mistap-proof.** Alle knoppen Engels en consistent (*doe nu* = ORDER/GO/YES/NEW/PULL/PRINT · *later* = WAIT · *weiger* = NO). **CLOSE TABLE** staat apart, rood en met bevestiging. Elke tik geeft directe feedback.

2. Kelnerselectie (routing)

De kern: **elke CTA gaat naar precies één handy** — die van de kelner die volgens Oscar de tafel bedient. Hoe weet Oscar dat? Via **wijken** en **POS-activiteit**, volledig automatisch.

Wijken

- Een **wijk** is een vast bereik tafelnummers, vooraf ingesteld per locatie. Voorbeeld: wijk "RES Noord" = tafels 1–19.
- Eigenaarschap geldt op **wijk-niveau**: de eigenaar van een wijk krijgt de CTA's van alle tafels in dat bereik.

Wie is de eigenaar?

- De **eerste kelner die in een wijk aanslaat** (POS) wordt vanaf dat moment eigenaar van de hele wijk.
- Het eigenaarschap **verschuift** automatisch zodra een andere kelner de meerderheid ($\pm 70\%$) van de recente tickets in die wijk heeft aangeslagen, binnen een tijdvenster. Eén losse aanslag (een collega die bijspringt) kapt de wijk dus niet over.
- Is er **nog geen eigenaar** (niemand heeft in de wijk aangeslagen), dan vervalt een CTA voor die wijk. Bewust geaccepteerd: de nullijn is nul meldingen, dus een gemiste melding is geen verslechtering — een verkeerd geroute melding wél.

Schoon houden — CTA 7. Om te voorkomen dat CTA's blijven gaan naar een kelner die weg of op pauze is, vraagt Oscar bij langere inactiviteit: "*Ben jij nog actief in deze wijk?*" Antwoordt de kelner NEE of PAUZE, dan gaat hij uit het kelneroverzicht en routeert Oscar niet meer naar hem.

Randgeval — eerste tafel van de wijk. CTA 1 (eerste drankje) kan vuren vóór er ook maar iets is aangeslagen in een verse wijk. Dan is er nog geen eigenaar en vervalt die ene CTA. Bewust geaccepteerd; geen seeding.

3. Druk

"Druk" = de **actuele werkdruk nu**: de order-rate (POS-orders per tijdseenheid over een recent venster) ligt boven een drempel. Gebruikt door CTA 5 (geen gift tijdens drukte) en als druktemeter op het LG-dashboard.

De druktemeter loopt vanzelf terug als de zaak leegloopt — een nalopende tafel op een uitgelopen avond houdt "druk" dus niet kunstmatig hoog. (CTA 3 gebruikt géén drukte, maar puur "seater actief".)

4. De CTA's

CTA 1 Eerste drankje

Doel: de eerste bestelling binnenhalen. **Trigger:** een ingestelde tijd na seaten zonder POS-order.

Sluit automatisch zodra er een POS-order komt (via welke weg dan ook). Zolang er niets besteld is, komt hij meerdere keren terug, tot de eerste order binnen is of de levensduur verloopt. Daarna neemt CTA 2 het over.

ORDER WAIT MOVE CLOSE TABLE

CLOSE TABLE staat apart en rood + vraagt een bevestiging — beschermd tegen mistaps.

CTA 2 Sleeping table

Doel: een tafel die stilligt oppikken. **Trigger:** geen kelner-actie gedurende de ingestelde tijd. **Terugkerend.**

De timer reset na elke kelner-actie op de tafel: een POS-aanslag óf een "wil nog wachten"-registratie. Meet vanaf de laatste actie, niet vanaf seaten — typisch ná het eten (tijd voor dessert, koffie, rekening).

ORDER WACHT VERPLAATS MELD LG

MELD LG is puur informatief; de tafel blijft open.

CTA 3 Seater actief (why not seated?)

Doel: tafels vangen die buiten de seater om geopend worden. **Trigger:** een kelner opent een niet-geseate tafel terwijl de seater actief is (een seat-actie binnen de laatste X minuten).

Blokkerende keuzevraag: de tafel gaat pas open ná de keuze.

NEW PULL

NEW = nieuwe gasten. PULL = gasten komen van een andere tafel → "Pull table: [oude tafel] → deze tafel", Oscar haalt bon + seat over, de oude tafel komt vrij.

CTA 4 Aanbieden (promo — achtergrond)

Doel: offreren toestaan op een ticket met een afgesproken aanbieding. **Trigger:** aan het reserveringsticket hangt de tag "promo".

Dit draait **op de achtergrond: iedere kelner** mag voor dit ticket gerechten en drank offreren — geen kaart, geen actie nodig; het recht staat automatisch aan zolang de "promo"-tag erop zit. Geen rol-rechten of blokkade: de aanbieding is immers al goedgekeurd op de reservering.

CTA 5 Loyalty — bied koffie aan

Doel: een gulle tafel belonen met een kleine attentie (gratis koffie). **Trigger:** *niet druk én besteding per persoon ≥ een instelbaar bedrag* (één getal per locatie).

Bewust simpel: géén percentielen, rolling averages, tijdvakken of vaste eval-tijd — dat was overbodige complexiteit voor een gratis koffie. Cooldown geldt per herkende gast (reservering/Amigo).

ORDER WAIT NO

WAIT stelt uit (komt later terug). NO = de kelner vindt het niet nodig → CTA 5 vervalt voor dit ticket.

CTA 6 Ready for next course (hoofdgerecht)

Doel: het hoofdgerecht op tijd doorzetten. **Trigger:** een ingestelde tijd (default 20 min) ná het voorgerecht. Kaart: "Tafel [nr] — Ready for next course?"

GO NO

GO → hoofdgerecht firen ("kan door"). NO → uitstellen; komt terug, **max 2× per ticket** (geen loop). Na de 2e NO firet het hoofdgerecht automatisch na 5 min, zodat de keuken niet blijft hangen.

CTA 7 Actief-check kelner

Doel: het kelneroverticht en daarmee de routing schoon houden. **Trigger:** een kelner is langere tijd inactief in zijn wijk.

Kaart: "Hoi [kelner], ben jij nu nog steeds actief in wijk [wijk]?"

YES NO BREAK

YES = blijft eigenaar. NO en BREAK = uit het kelneroverticht (geen routing meer naar hem). **Geen antwoord** → **fail-safe**: de kelner blijft behouden; Oscar vraagt later gewoon opnieuw (nooit droppen op een gemist tikje).

CTA 8 Ready for dessert

Doel: het dessert doorzetten. **Trigger:** een ingestelde tijd (default 20 min) ná het firen van het hoofdgerecht. Zelfde principe als CTA 6, maar voor het dessert. Kaart: "Tafel [nr] — Ready for dessert?"

GO NO

GO → dessert firen ("kan door"). NO → komt nog één keer terug (max 2× per ticket, geen loop).

CTA 9 Gast wil bestellen

Doel: een bestelverzoek van de gast bij de kelner krijgen. **Trigger:** een **runner** (geen kelner) geeft via het externe **iPad-scherm** door dat een tafel wil bestellen — de gast vraagt het aan de runner, die tikt op de iPad de tafel + "bestellen". Oscar pusht naar de kelner van de wijk.

ORDER NO

ORDER → open WaiterPro en neem de bestelling op. NO → wegglikken. **Status gaat terug naar de runner-iPad** ("opgepakt"/"afgewezen") — geen dubbele kaarten, geen wachtende gast.

CTA 10 Gast wil betalen

Doel: een betaalverzoek bij de kelner krijgen. **Trigger:** zoals CTA 9, maar de runner kiest op de iPad "betalen".

PRINT NO

PRINT → reken af / druk de bon af. NO → wegglikken. Status gaat terug naar de runner-iPad ("klaar"/"afgewezen").

5. Logging & database

Elke afgeronde CTA — een kelner-respons (knop) óf een verval — wordt vastgelegd in twee lagen:

- **Live log (backend):** rollend overzicht van de laatste ~500 regels, voor snelle monitoring.
- **Database (persistent):** elke regel wordt apart ge-insert. Volledige historie, als basis voor het LG-dashboard en analyses (response-tijden, CTA-volume per kelner/locatie/tijdvak, hoe vaak CTA's vervallen).

Record-formaat: `cta_nr;datum;tijd;kelner;tafelnr;status;actie;response_sec` — bijv. `1;2026-06-12;19:42:13;Peter;14;enabled;ORDER;12`.

Elke CTA is per locatie **enabled**, **disabled** of **deleted**. Staat een CTA op **disabled**, dan verschijnt hij niet op de handy maar wordt elke keer dat hij *zou* afgaan tóch gelogd (zonder respons) — zo zie je hoe vaak hij zou triggeren. Over tijd laat dat zien of **kelnors beter worden** (minder zou-fires, of sneller/zelf opgelost na correctie). **deleted** logt niets.

Oscar CTA-systeem · interne uitleg · juni 2026. Volledige specificatie met parameters en beslislog: `oscar_cta_specs.md`. Interactieve demo: github.com/peterstuurman-hash/CTA-